



Wytyczne do projektowania układów cyfrowych w języku SystemVerilog

Materiały dodatkowe dla studentów kierunku Mikro-
elektronika w Technice i Medycynie (AGH)

dr inż. Paweł Skrzypiec
mgr inż. Łukasz Kadłubowski
dr hab. inż. Robert Szczygieł

Kraków, 5 października 2025

Spis treści

Spis kodów źródłowych	3
1 Styl kodowania	4
1.1 Formatowanie ogólne	4
1.2 Spacje, nawiasy i operatory	5
1.3 Bloki i struktura	6
1.4 Konstrukcje sekwencyjne i kombinacyjne	7
1.5 Nazewnictwo	8
1.6 Literały i liczby	9
1.7 Moduły, porty, parametry, instancje	9
1.8 Pakiety i importy	10
1.9 Komentarze i dokumentacja	10
1.10 Maszyny stanów skończonych	11
1.11 Organizacja projektu	12
2 Przykładowe układy cyfrowe	13
2.1 Bramki logiczne	13
2.2 Detektory zbocza	16
2.3 Dzielniki sygnału zegarowego	20
2.4 Rejestry przesuwne	24
2.5 Liczniki	28
2.6 Maszyny stanów skończonych	32
2.7 Serializery	37
2.8 Deserializery	42
2.9 Pamięci	47

Spis kodów źródłowych

List. 2.1	Moduł bramki and2.	13
List. 2.2	Test modułu bramki and2.	14
List. 2.3	Moduł detektora zbocza narastającego.	16
List. 2.4	Test modułu detektora zbocza narastającego.	17
List. 2.5	Moduł układu dzielącego częstotliwość przez 2.	20
List. 2.6	Test modułu układu dzielącego częstotliwość przez 2.	21
List. 2.7	Moduł rejestru przesuwającego dane w prawo.	24
List. 2.8	Test modułu rejestru przesuwającego dane w prawo.. . . .	25
List. 2.9	Moduł 32-bitowego licznika.	28
List. 2.10	Test modułu 32-bitowego licznika.	29
List. 2.11	Moduł przykładowej maszyny stanów skończonych.	32
List. 2.12	Test modułu przykładowej maszyny stanów skończonych.	34
List. 2.13	Moduł serializera 8 do 1.	37
List. 2.14	Test modułu serializera 8 do 1.	39
List. 2.15	Moduł deserializera 1 do 8.	42
List. 2.16	Test modułu deserializera 1 do 8.	44
List. 2.17	Moduł pamięci RAM 1024 × 32b z synchronicznym odczytem.	47
List. 2.18	Test modułu pamięci RAM 1024 × 32b z synchronicznym odczytem. . .	48

1. Styl kodowania

1.1. Formatowanie ogólne

Maksymalna długość linii

- Maksymalna długość linii nie powinna przekroczyć **120** znaków.

Wcięcia

- Każdy poziom zagnieżdżenia powinien być zbudowany z **4** spacji.
- Instrukcje pierwszego poziomu zagnieżdżenia znajdujące się wewnątrz modułu, interfejsu, pakietu (`package`), itp., nie powinny posiadać wcięć.

Ostatnia nowa linia

- Zalecane jest zostawienie jednej pustej linii na końcu każdego pliku.

Zbędne spacje na końcu linii

- Na końcu linii nie powinna znajdować się żadna spacja.

Jednostka czasu

- Definiowanie jednostki czasu na początku pliku nie jest wymagane.

Jeden moduł / interfejs / klasa na plik

- W pliku zdefiniowany może zostać tylko jeden moduł, interfejs albo klasa.
- Nazwa pliku powinna odpowiadać nazwie definiowanego modułu, interfejsu albo klasy.

Jedno wyrażenie na linię

- W linii umieszczone może być tylko jedno wyrażenie.

1. Styl kodowania

- Przykład źle napisanego kodu:

```
rvalid = 1'b1; rdata = 32'h0;
```

1.2. Spacje, nawiasy i operatory

Słowa kluczowe i nawiasy

- Pomiedzy słowami kluczowymi a nawiasami powinny być umieszczane pojedyncze spacje.
- Przykład:

```
if (warunek)
```

Spacje wewnątrz nawiasów

- Wewnątrz nawiasów nie powinny być umieszczane spacje.
- Przykład dobrze napisanego kodu:

```
if (a == b)
```

- Przykład źle napisanego kodu:

```
if ( a == b )
```

Odstępy wokół operatorów

- Pomiedzy operatorami a operandami powinny być umieszczane spacje.
- Przykład dobrze napisanego kodu:

```
a + b
```

- Przykład źle napisanego kodu:

```
a+b
```

Spacje po przecinkach i średnikach

- Po przecinkach i średnikach występujących w wyrażeniach powinny być umieszczane pojedyncze spacje (nie dotyczy końca linii).
- Przykład:

```
a, b
```

1.3. Bloki i struktura

begin-end przy pojedynczym poleceniu

- Słowa kluczowe `begin` oraz `end` powinny być używane nawet przy pojedynczym poleceniu.
- Przykład:

```
if (valid) begin
    ++counter;
end
```

Styl instrukcji warunkowej if-else

- Jeżeli w którejkolwiek gałęzi wyrażenia `if-else` występują co najmniej dwa polecenia, słowa kluczowe `begin` oraz `end` powinny być umieszczone w każdej gałęzi.
- Słowo kluczowe `else` powinno być umieszczone w tej samej linii, co słowo kluczowe `end`.
- Przykład:

```
if (rvalid) begin
    dvalid_nxt = 1'b1;
    dout_nxt = rdata;
end else begin
    dout_nxt = 32'h0;
end
```

Styl instrukcji warunkowej case

- W bloku `case` etykiety powinny być poziom wcięcia głębiej w stosunku do słowa kluczowego `case`.
- Przykład:

```
case (state)
    IDLE: begin
        if (trigger) begin
            state_nxt = ACTIVE;
        end
    end
    ACTIVE: begin
        state_nxt = IDLE;
    end
endcase
```

1. Styl kodowania

- Stosowanie etykiety `default` jest obowiązkowe w blokach kombinacyjnych. Wyjątkiem są sytuacje, w których wewnątrz bloku kombinacyjnego zdefiniowane zostały przypisania domyślne.

Kolejność sekcji w module

- Wewnątrz modułów zalecana jest następująca kolejność sekcji:
 1. importy pakietów,
 2. parametry modułu,
 3. porty modułu,
 4. definicje parametrów lokalnych,
 5. definicje typów,
 6. definicje sygnałów lokalnych,
 7. przypisania (`assign`),
 8. instancje podmodułów,
 9. bloki sekwencyjne i kombinacyjne.

Nazwy i etykiety bloków

- Przypisywanie etykiet do bloków jest niezalecane.

1.4. Konstrukcje sekwencyjne i kombinacyjne

Specjalizowane bloki `always`

- Wymagane jest stosowanie specjalizowanych bloków `always_ff`, `always_comb` i `always_latch` zamiast generycznego bloku `always`.
- Użycie bloku `always` jest dozwolone jedynie w modułach symulacyjnych.

Przypisania

- W blokach sekwencyjnych stosować należy jedynie przypisania nieblokujące (`<=`), a w blokach kombinacyjnych przypisania blokujące (`=`).

Sygnał zegarowy

- Powinien posiadać nazwę `clk`.
- Powinien być aktywny zboczem narastającym.

Sygnał resetujący

- Wymagane jest stosowanie resetu asynchronicznego aktywnego stanem niskim o nazwie `rst_n`.

Inicjalizacja rejestrów

- Wszystkie rejestry powinny być inicjalizowane poprzez reset.
- Zabronione jest stosowanie konstrukcji `initial`.

1.5. Nazewnictwo

Stosowana notacja

- Wszelkie nazwy powinny być wzorowane na notacji *snake_case*.
- Przykład sygnału:

```
logic [7:0] bytes_counter;
```

- Przykład modułu:

```
module edge_detector (...);
```

Wielkość liter

- Wszystkie nazwy poza stałymi powinny być zapisywane małymi literami.
- Nazwy stałych powinny być zapisywane wielkimi literami.
- Przykład stałej:

```
const int BITS_IN_BYTE = 8;
```

- Przykład zmiennej:

```
logic [BITS_IN_BYTE-1:0] received_byte;
```


1. Styl kodowania

Sufiksy

- Sygnały, których wartości wyznaczane są kombinacyjnie w celu późniejszego przypisania do rejestrów powinny posiadać sufix `_nxt`.
- Sygnały aktywne stanem niskim powinny posiadać sufix `_n`.
- Zarówno definiowane struktury jak i typy wyliczeniowe powinny posiadać w nazwach sufix `_t`.

1.6. Literały i liczby

Szerokość literałów liczbowych

- Każdy definiowany literał liczbowy powinien mieć precyzyjnie określoną szerokość.
- Przykład:

```
counter = 32'h0;
```

Separatory

- W przypadku długich liczb zalecane jest stosowanie separatorów.
- Przykład:

```
const int FREQUENCY = 32'd100_000_000;
```

1.7. Moduły, porty, parametry, instancje

Format listy portów

- W linii zdefiniowany może być tylko jeden port.

Kolejność portów

1. Zegar,
2. Reset,
3. Wyjścia,
4. Wejścia.

Parametry

- Nazwy parametrów powinny być pisane wielkimi literami.

Instancje

- Sygnały powinny być podłączane do modułów przy wykorzystaniu połączeń nazwanych.
- Przykład:

```
my_module u_my_module (  
    .clk,  
    .rst_n,  
  
    .din(rdata)  
);
```

Testowany moduł

- Testowany moduł zawsze powinien posiadać nazwę `dut`.

1.8. Pakiety i importy

Wykorzystywanie pakietów

- Zalecane jest umieszczanie parametrów współdzielonych pomiędzy modułami wewnątrz pakietów.

Importowanie zawartości pakietów

- Instrukcje importujące zawartość wykorzystywanych pakietów powinna być umieszczona pomiędzy nazwą modułów a listą jego portów.
- Przykład:

```
module alu  
    import core_pkg::*;  
(
```

1.9. Komentarze i dokumentacja

1. Styl kodowania

Komentarze

- Komentarze powinny być wykorzystywane jedynie do tłumaczenia nieintuicyjnych fragmentów kodu, np. wyjaśniania dlaczego wykorzystany został taki a nie inny algorytm do rozwiązania danego problemu.
- Zarówno komentarze jednowierszowe jak wielowierszowe powinny stosować notację komentarza blokowego `/* */`.
- Przykład komentarza jednowierszowego:

```
/* komentarz jednowierszowy */
```

- Przykład komentarza wielowierszowego:

```
/**  
 * Komentarz  
 * wielowierszowy  
 */
```

1.10. Maszyny stanów skończonych

Definiowanie stanów FSM

- Stany maszyny stanów powinny być definiowane przy wykorzystaniu dedykowanego typu wyliczeniowego.
- Przykład:

```
typedef enum logic {  
    IDLE,  
    ACTIVE  
} state_t;  
  
state_t state, state_nxt;
```

Rozdzielanie logiki sekwencyjnej i kombinacyjnej

- Zalecane jest rozdzielanie logiki sekwencyjnej i kombinacyjnej.
- Blok kombinacyjny powinien być odpowiedzialny za wyznaczanie kolejnych stanów,
- Blok sekwencyjny powinien zawierać jedynie przypisywanie wartości wyznaczonych w bloku kombinacyjnym do odpowiednich przerzutników.

1.11. Organizacja projektu

Struktura repozytorium

- W repozytorium utworzone powinny zostać następujące katalogi:
 - `constraints` - ograniczenia projektowe,
 - `ip` - istniejące komponenty,
 - `rtl` - pliki syntezywalne,
 - `sim` - pliki symulacyjne.

2. Przykładowe układy cyfrowe

2.1. Bramki logiczne

Przykładowy moduł

List. 2.1 Moduł bramki and2.

```
/* Copyright (C) 2025 AGH University of Krakow */

module and2 (
    output logic y,
    input logic a,
    input logic b
);

/* Signals assignments */

assign y = a & b;

endmodule
```

Przykładowa symulacja

List. 2.2 Test modułu bramki and2.

```
/* Copyright (C) 2025 AGH University of Krakow */

module tb_and2;

/* Local variables and signals */

logic y, a, b;

/* Submodules placement */

and2 dut (
    .y,
    .a,
    .b
);

/* Test */

initial begin
    a = 1'b0;
    b = 1'b0;

    for (int i = 0; i < 4; ++i) begin
        a = i[0];
        b = i[1];
        #1;

        assert (y == (a & b)) else
            $error("y: exp: 0b%b, rcv: 0b%b", a & b, y);
    end

    $finish;
end

endmodule
```

2. Przykładowe układy cyfrowe

Zadania

1. Zaprojektuj moduł oraz opracuj symulację układu implementującego bramkę:
 - (a) or2,
 - (b) xor2,
 - (c) nand2,
 - (d) nor2,
 - (e) xnor2.

2.2. Detektory zbocza

Przykładowy moduł

List. 2.3 Moduł detektora zbocza narastającego.

```
/* Copyright (C) 2025 AGH University of Krakow */

module rising_edge_detector (
    input logic clk,
    input logic rst_n,

    output logic edge_detected,
    input logic din
);

/* Local variables and signals */
logic din_buf;

/* Module internal logic */
always_ff @(posedge clk or negedge rst_n) begin
    if (!rst_n)
        din_buf <= 1'b0;
    else
        din_buf <= din;
end

always_comb
    edge_detected = din & ~din_buf;

endmodule
```


2. Przykładowe układy cyfrowe

Przykładowa symulacja

List. 2.4 Test modułu detektora zbocza narastającego.

```
/* Copyright (C) 2025 AGH University of Krakow */

module tb_rising_edge_detector;

    /* Local variables and signals */

    logic clk, rst_n;

    logic din, edge_detected;

    /* Submodules placement */

    rising_edge_detector dut (
        .clk,
        .rst_n,

        .edge_detected,
        .din
    );

    /* Tasks and functions definitions */

    task reset();
        for (int i = 0; i < 2; ++i) begin
            @(negedge clk);
            rst_n = i[0];
        end
    endtask

    task test_constant_signal(logic val);
        din = val;

        reset();

        @(negedge clk);

        assert (!edge_detected) else
            $error("edge_detected: exp: 0b%b, rcv: 0b%b", 1'b0, edge_detected);

        din = 1'b0;
    endtask

    task test_non_constant_signal(logic val);
        din = val;
```

```
reset();

@(negedge clk);
din = ~val;
#1;

assert (edge_detected == (val == 1'b0)) else
    $error("edge_detected: exp: 0b%b, rcv: 0b%b", (val == 1'b0), edge_detected);

din = 1'b0;
endtask

/* Clock generation */

initial begin
    clk = 1'b0;

    forever
        clk = #5ns ~clk;
end

/* Test */

initial begin
    rst_n = 1'b1;
    din = 1'b0;

    for (int i = 0; i < 2; ++i)
        test_constant_signal(i[0]);

    for (int i = 0; i < 2; ++i)
        test_non_constant_signal(i[0]);

    $finish;
end

endmodule
```

2. Przykładowe układy cyfrowe

Zadania

1. Zaprojektuj moduł oraz opracuj symulację układu implementującego:
 - (a) detektor zbocza opadającego,
 - (b) detektor obu zboczy (narastającego i opadającego).

2.3. Dzielniki sygnału zegarowego

Przykładowy moduł

List. 2.5 Moduł układu dzielącego częstotliwość przez 2.

```
/* Copyright (C) 2025 AGH University of Krakow */

module clock_divider (
    input logic clk,
    input logic rst_n,

    output logic clk_out
);

/* Module internal logic */

always_ff @(posedge clk or negedge rst_n) begin
    if (!rst_n)
        clk_out <= 1'b0;
    else
        clk_out <= ~clk_out;
end

endmodule
```

2. Przykładowe układy cyfrowe

Przykładowa symulacja

List. 2.6 Test modułu układu dzielącego częstotliwość przez 2.

```
/* Copyright (C) 2025 AGH University of Krakow */

module tb_clock_divider;

    /* Local variables and signals */

    logic clk, rst_n;

    logic clk_out;

    /* Submodules placement */

    clock_divider dut (
        .clk,
        .rst_n,

        .clk_out
    );

    /* Tasks and functions definitions */

    task reset();
        for (int i = 0; i < 2; ++i) begin
            @(negedge clk);
            rst_n = i[0];
        end
    endtask

    task test_clock_division();
        reset();

        for (int i = 0; i < 10; ++i) begin
            assert (clk_out == i[0]) else
                $error("clk_out: exp: 0b%b, rcv: 0b%b", i[0], clk_out);
            @(negedge clk);
        end
    endtask

    /* Clock generation */

    initial begin
        clk = 1'b0;

        forever
```

2. Przykładowe układy cyfrowe

```
        clk = #5ns ~clk;
end

/* Test */

initial begin
    rst_n = 1'b1;

    test_clock_division();

    $finish;
end

endmodule
```

2. Przykładowe układy cyfrowe

Zadania

1. Zaprojektuj moduł oraz opracuj symulację układu:
 - (a) dzielącego częstotliwość sygnału zegarowego przez 3,
 - (b) który na podstawie stanu portu wejściowego divider dzieli częstotliwość przez współczynnik z zakresu 2 - 5.

2.4. Rejestry przesuwne

Przykładowy moduł

List. 2.7 Moduł rejestru przesuwającego dane w prawo.

```
/* Copyright (C) 2025 AGH University of Krakow */

module shift_register (
    input logic clk,
    input logic rst_n,

    output logic dout,
    input logic din
);

/* Local variables and signals */

logic [3:0] data;

/* Signals assignments */

assign dout = data[0];

/* Module internal logic */

always_ff @(posedge clk or negedge rst_n) begin
    if (!rst_n)
        data <= 4'h0;
    else
        data <= {din, data[3:1]};
end

endmodule
```


2. Przykładowe układy cyfrowe

Przykładowa symulacja

List. 2.8 Test modułu rejestru przesuwającego dane w prawo..

```
/* Copyright (C) 2025 AGH University of Krakow */

module tb_shift_register;

    /* Local variables and signals */

    logic clk, rst_n;

    logic din, dout;

    /* Submodules placement */

    shift_register dut (
        .clk,
        .rst_n,

        .dout,
        .din
    );

    /* Tasks and functions definitions */

    task reset();
        for (int i = 0; i < 2; ++i) begin
            @(negedge clk);
            rst_n = i[0];
        end
    endtask

    task test_data_shifting();
        logic [3:0] wdata, rdata;

        wdata = $random();

        reset();

        fork
            begin
                for (int i = 0; i < 4; ++i) begin
                    @(negedge clk);
                    din = wdata[i];
                end
            end
            begin
                for (int i = 0; i < 4; ++i)
```

```
        @(negedge clk);

        for (int i = 0; i < 4; ++i) begin
            @(negedge clk);
            rdata[i] = dout;
        end
    end
join

    assert (rdata == wdata) else
        $error("rdata: exp: 0x%x, rcv: 0x%x", wdata, rdata);
endtask

/* Clock generation */

initial begin
    clk = 1'b0;

    forever
        clk = #5ns ~clk;
end

/* Test */

initial begin
    rst_n = 1'b1;

    test_data_shifting();

    $finish;
end

endmodule
```

2. Przykładowe układy cyfrowe

Zadania

1. Zaprojektuj moduł oraz opracuj symulację układu implementującego rejestr przesuw-ny:
 - (a) przesuwający dane w lewo,
 - (b) z równoległym ładowaniem, przesuwający dane w prawo,
 - (c) z równoległym ładowaniem, w którym kierunek przesuwania będzie ustalany na podstawie stanu portu wejściowego `rshift`.

2.5. Liczniki

Przykładowy moduł

List. 2.9 Moduł 32-bitowego licznika.

```
/* Copyright (C) 2025 AGH University of Krakow */

module counter (
    input logic clk,
    input logic rst_n,

    input logic enabled,

    output logic [31:0] value
);

/* Local variables and signals */

logic [31:0] value_nxt;

/* Module internal logic */

always_ff @(posedge clk or negedge rst_n) begin
    if (!rst_n)
        value <= 32'b0;
    else
        value <= value_nxt;
end

always_comb begin
    value_nxt = value;

    if (enabled)
        value_nxt = value + 1;
end

endmodule
```

2. Przykładowe układy cyfrowe

Przykładowa symulacja

List. 2.10 Test modułu 32-bitowego licznika.

```
/* Copyright (C) 2025 AGH University of Krakow */

module tb_counter;

    /* Local variables and signals */

    logic clk, rst_n;

    logic [31:0] value;
    logic enabled;

    /* Submodules placement */

    counter dut (
        .clk,
        .rst_n,

        .enabled,

        .value
    );

    /* Tasks and functions definitions */

    task reset();
        for (int i = 0; i < 2; ++i) begin
            @(negedge clk);
            rst_n = i[0];
        end
    endtask

    task test_disabled_counter();
        enabled = 1'b0;

        reset();

        for (int i = 0; i < 10; ++i) begin
            assert (value == 32'b0) else
                $error("value: exp: %1d, rcv: %1d", 32'b0, value);
            @(negedge clk);
        end
    endtask

    task test_enabled_counter();
        enabled = 1'b1;
```

```
    reset();

    for (int i = 0; i < 10; ++i) begin
        assert (value == i) else
            $error("value: exp: %1d, rcv: %1d", i, value);
        @(negedge clk);
    end
endtask

/* Clock generation */

initial begin
    clk = 1'b0;

    forever
        clk = #5ns ~clk;
end

/* Test */

initial begin
    rst_n = 1'b1;
    enabled = 1'b0;

    test_disabled_counter();
    test_enabled_counter();

    $finish;
end

endmodule
```

2. Przykładowe układy cyfrowe

Zadania

1. Zaprojektuj moduł oraz opracuj symulację układu implementującego:
 - (a) 32-bitowy licznik zliczający w dół,
 - (b) 32-bitowy licznik zliczający w obu kierunkach (kierunek zliczania powinien być zależny stanu portu wejściowego `increment`).

2.6. Maszyny stanów skończonych

Przykładowy moduł

List. 2.11 Moduł przykładowej maszyny stanów skończonych.

```
/* Copyright (C) 2025 AGH University of Krakow */

module fsm (
    input logic clk,
    input logic rst_n,

    output logic done,
    input logic trigger
);

/* User defined types and constants */

typedef enum logic {
    IDLE,
    ACTIVE
} state_t;

/* Local variables and signals */

state_t state, state_nxt;
logic [3:0] counter, counter_nxt;

/* Module internal logic */

always_ff @(posedge clk or negedge rst_n) begin
    if (!rst_n) begin
        state <= IDLE;
        counter <= 4'b0;
    end else begin
        state <= state_nxt;
        counter <= counter_nxt;
    end
end

always_comb begin
    state_nxt = state;
    counter_nxt = counter;

    case (state)
        IDLE: begin
            if (trigger)
                state_nxt = ACTIVE;
        end
    endcase
end
```


2. Przykładowe układy cyfrowe

```
    end
    ACTIVE: begin
        if (counter == 10) begin
            state_nxt = IDLE;
            counter_nxt = 4'b0;
        end else begin
            counter_nxt = counter + 1;
        end
    end
endcase
end

always_comb begin
    done = 1'b0;

    case (state)
        IDLE: ;
        ACTIVE: begin
            if (counter == 10)
                done = 1'b1;
            end
        endcase
    end

end

endmodule
```

Przykładowa symulacja

List. 2.12 Test modułu przykładowej maszyny stanów skończonych.

```
/* Copyright (C) 2025 AGH University of Krakow */

module tb_fsm;

    /* Local variables and signals */

    logic clk, rst_n;

    logic trigger, done;

    /* Submodules placement */

    fsm dut (
        .clk,
        .rst_n,

        .done,
        .trigger
    );

    /* Tasks and functions definitions */

    task reset();
        for (int i = 0; i < 2; ++i) begin
            @(negedge clk);
            rst_n = i[0];
        end
    endtask

    task test_non_triggered_fsm();
        for (int i = 0; i < 15; ++i) begin
            assert (!done) else
                $error("done: exp: %b, rcv: %b", 1'b0, done);
            @(negedge clk);
        end
    endtask

    task test_triggered_fsm();
        @(negedge clk);
        trigger = 1'b1;

        fork
            begin
                @(negedge clk);
                trigger = 1'b0;
            end
        join
    endtask

endmodule
```

2. Przykładowe układy cyfrowe

```
end
begin
    for (int i = 0; i < 10; ++i) begin
        @(negedge clk);
        assert (!done) else
            $error("done: exp: %b, rcv: %b", 1'b0, done);
    end

    @(negedge clk);
    assert (done) else
        $error("done: exp: %b, rcv: %b", 1'b1, done);
end

join
endtask

/* Clock generation */

initial begin
    clk = 1'b0;

    forever
        clk = #5ns ~clk;
end

/* Test */

initial begin
    trigger = 1'b0;

    reset();

    test_non_triggered_fsm();
    test_triggered_fsm();

    $finish();
end

endmodule
```

Zadania

1. Zaprojektuj moduł oraz opracuj symulację układu, który po 15 taktach od wystąpienia wysokiego stanu logicznego na wejściowym porcie `trigger` ustawi na 2 takty zegara wysoki stan logiczny na porcie wyjściowym `done`.

2. Przykładowe układy cyfrowe

2.7. Serializery

Przykładowy moduł

List. 2.13 Moduł serializera 8 do 1.

```
/* Copyright (C) 2025 AGH University of Krakow */

module serializer (
    input logic clk,
    input logic rst_n,

    input logic wvalid,
    input logic [7:0] wdata,

    output logic dout
);

/* User defined types and constants */

typedef enum logic {
    IDLE,
    ACTIVE
} state_t;

/* Local variables and signals */

state_t state, state_nxt;
logic [2:0] counter, counter_nxt;

/* Module internal logic */

always_ff @(posedge clk or negedge rst_n) begin
    if (!rst_n) begin
        state <= IDLE;
        counter <= 3'b0;
    end else begin
        state <= state_nxt;
        counter <= counter_nxt;
    end
end

always_comb begin
    state_nxt = state;
    counter_nxt = counter;

    case (state)
        IDLE: begin
```

```
        if (wvalid)
            state_nxt = ACTIVE;
        end
        ACTIVE: begin
            if (counter == 7) begin
                state_nxt = IDLE;
                counter_nxt = 3'b0;
            end else begin
                counter_nxt = counter + 1;
            end
        end
    endcase
end

always_comb begin
    dout = 1'b0;

    case (state)
        IDLE: ;
        ACTIVE: begin
            dout = wdata[counter];
        end
    endcase
end

endmodule
```

2. Przykładowe układy cyfrowe

Przykładowa symulacja

List. 2.14 Test modułu serializera 8 do 1.

```
/* Copyright (C) 2025 AGH University of Krakow */

module tb_serializer;

    /* Local variables and signals */

    logic clk, rst_n;

    logic [7:0] wdata;
    logic wvalid;

    logic dout;

    /* Submodules placement */

    serializer dut (
        .clk,
        .rst_n,

        .wvalid,
        .wdata,

        .dout
    );

    /* Tasks and functions definitions */

    task reset();
        for (int i = 0; i < 2; ++i) begin
            @(negedge clk);
            rst_n = i[0];
        end
    endtask

    task test_data_serialization();
        logic [7:0] rdata;

        reset();

        fork
            begin
                wvalid = 1'b1;
                wdata = $random();

                @(negedge clk);
```

```
        wvalid = 1'b0;
    end
    begin
        for (int i = 0; i < 8; ++i) begin
            @(negedge clk);
            rdata[i] = dout;
        end
    end
    join

    assert (rdata == wdata) else
        $error("rdata: exp: 0x%x, rcv: 0x%x", wdata, rdata);

    wdata = 8'h0;
endtask

/* Clock generation */

initial begin
    clk = 1'b0;

    forever
        clk = #5ns ~clk;
end

/* Test */

initial begin
    rst_n = 1'b1;
    wvalid = 1'b0;
    wdata = 8'b0;

    test_data_serialization();

    $finish;
end

endmodule
```


2. Przykładowe układy cyfrowe

Zadania

1. Zaprojektuj moduł oraz opracuj symulację układu implementującego serializer:
 - (a) 8 do 1, wysuwający dane od bitu najstarszego do najmłodszego,
 - (b) 8 do 1, wysuwający dane od bitu najstarszego do najmłodszego, w którym dane zostaną poprzedzone bitem startu ($1'b0$) i zakończone bitem stopu ($1'b1$).

2.8. Deserializery

Przykładowy moduł

List. 2.15 Moduł deserializera 1 do 8.

```
/* Copyright (C) 2025 AGH University of Krakow */

module deserializer (
    input logic clk,
    input logic rst_n,

    output logic rvalid,
    output logic [7:0] rdata,

    input logic din
);

/* User defined types and constants */

typedef enum logic {
    IDLE,
    ACTIVE
} state_t;

/* Local variables and signals */

state_t state, state_nxt;
logic [2:0] counter, counter_nxt;

logic [7:0] rdata_nxt;
logic rvalid_nxt;

/* Module internal logic */

always_ff @(posedge clk or negedge rst_n) begin
    if (!rst_n) begin
        state <= IDLE;
        counter <= 3'b0;
    end else begin
        state <= state_nxt;
        counter <= counter_nxt;
    end
end

always_comb begin
    state_nxt = state;
    counter_nxt = counter;
end
```

2. Przykładowe układy cyfrowe

```
    case (state)
        IDLE: begin
            /* start bit detection */
            if (!din)
                state_nxt = ACTIVE;
            end
        ACTIVE: begin
            if (counter == 7) begin
                state_nxt = IDLE;
                counter_nxt = 3'b0;
            end else begin
                counter_nxt = counter + 1;
            end
        end
    endcase
end

always_ff @(posedge clk or negedge rst_n) begin
    if (!rst_n) begin
        rvalid <= 1'b0;
        rdata <= 8'h0;
    end else begin
        rvalid <= rvalid_nxt;
        rdata <= rdata_nxt;
    end
end

always_comb begin
    rvalid_nxt = 1'b0;
    rdata_nxt = rdata;

    case (state)
        IDLE: ;
        ACTIVE: begin
            rdata_nxt[counter] = din;

            if (counter == 7)
                rvalid_nxt = 1'b1;
        end
    endcase
end

endmodule
```

Przykładowa symulacja

List. 2.16 Test modułu deserializera 1 do 8.

```
/* Copyright (C) 2025 AGH University of Krakow */

module tb_deserializer;

    /* Local variables and signals */

    logic clk, rst_n;

    logic [7:0] rdata;
    logic rvalid;

    logic din;

    /* Submodules placement */

    deserializer dut (
        .clk,
        .rst_n,

        .rvalid,
        .rdata,

        .din
    );

    /* Tasks and functions definitions */

    task reset();
        for (int i = 0; i < 2; ++i) begin
            @(negedge clk);
            rst_n = i[0];
        end
    endtask

    task test_data_deserialization();
        logic [7:0] wdata = $random();

        reset();

        fork
            begin
                din = 1'b0;

                for (int i = 0; i < 8; ++i) begin
                    @(negedge clk);
```

2. Przykładowe układy cyfrowe

```
        din = wdata[i];
    end

    @(negedge clk) ;
    din = 1'b1;
end
begin
    for (int i = 0; i < 9; ++i) begin
        assert (!rvalid) else
            $error("rvalid: exp: 0b%b, rcv: 0b%b", 1'b0, rvalid);
        @(negedge clk);
    end
end
join

assert (rvalid && (rdata == wdata)) else
    $error("rvalid: exp: 0b%b, rcv: 0b%b; rdata: exp: 0x%x, rcv: 0x%x",
        1'b0, rvalid, wdata, rdata);
endtask

/* Clock generation */

initial begin
    clk = 1'b0;

    forever
        clk = #5ns ~clk;
end

/* Test */

initial begin
    rst_n = 1'b1;
    din = 1'b1;

    test_data_deserialization();

    $finish;
end

endmodule
```

Zadania

1. Zaprojektuj moduł oraz opracuj symulację układu implementującego deserializer:
 - (a) 1 do 8, wsuwający dane od bitu najstarszego do najmłodszego, w którym dane będą poprzedzane bitem startu (1'b0),
 - (b) 1 do 8, wsuwający dane od bitu najstarszego do najmłodszego, w którym dane będą poprzedzone bitem startu (1'b0) i zakończone bitem stopu (1'b1).

2. Przykładowe układy cyfrowe

2.9. Pamięci

Przykładowy moduł

List. 2.17 Moduł pamięci RAM 1024 × 32b z synchronicznym odczytem.

```
/* Copyright (C) 2025 AGH University of Krakow */

module ram (
    input logic clk,

    output logic [31:0] rdata,
    input logic [11:0] address,
    input logic we,
    input logic [31:0] wdata
);

/* Local variables and signals */

logic [31:0] mem [1024];

/* Module internal logic */

always_ff @(posedge clk) begin
    if (we)
        mem[address[11:2]] <= wdata;

    rdata <= mem[address[11:2]];
end

endmodule
```

Przykładowa symulacja

List. 2.18 Test modułu pamięci RAM 1024 × 32b z synchronicznym odczytem.

```
/* Copyright (C) 2025 AGH University of Krakow */

module tb_ram;

    /* Local variables and signals */

    logic clk;

    logic [31:0] wdata, rdata;
    logic [11:0] address;
    logic we;

    /* Submodules placement */

    ram dut (
        .clk,

        .rdata,
        .address,
        .we,
        .wdata
    );

    /* Tasks and functions definitions */

    task write(logic [11:0] addr, logic [31:0] val);
        @(negedge clk);
        address = addr;
        we = 1'b1;
        wdata = val;

        @(negedge clk);
        address = 12'h0;
        we = 1'b0;
        wdata = 32'h0;
    endtask

    task read(output logic [31:0] val, input logic [11:0] addr);
        @(negedge clk);
        address = addr;

        @(negedge clk);
        val = rdata;
    endtask
endmodule
```


2. Przykładowe układy cyfrowe

```
        address = 12'h0;
    endtask

    task test_write();
        logic [31:0] val;
        logic [11:0] addr;

        addr = $random() & ~12'h3;
        val = $random();

        write(addr, val);

        assert (dut.mem[addr[11:2]] == val) else
            $error("dut.mem[%4d]: exp: 0x%x, rcv: 0x%x",
                addr[11:2], val, dut.mem[addr[11:2]]);
    endtask

    task test_write_read();
        logic [31:0] wval, rval;
        logic [11:0] addr;

        addr = $random() & ~12'h3;
        wval = $random();

        write(addr, wval);
        read(rval, addr);

        assert (rval == wval) else
            $error("rval: exp: 0x%x, rcv: 0x%x", wval, rval);
    endtask

    /* Clock generation */

    initial begin
        clk = 1'b0;

        forever
            clk = #5ns ~clk;
    end

    /* Test */

    initial begin
        address = 12'h0;
        we = 1'b0;
        wdata = 32'h0;

        test_write();
        test_write_read();

        $finish;
```

2. Przykładowe układy cyfrowe

```
end  
endmodule
```

2. Przykładowe układy cyfrowe

Zadania

1. Zaprojektuj moduł oraz opracuj symulację układu implementującego pamięć RAM:
 - (a) $1024 \times 32b$ z asynchronicznym odczytem,
 - (b) $128 \times 8b$ z synchronicznym odczytem,
 - (c) $1024 \times 32b$ z synchronicznym odczytem i sygnałem aktywującym poszczególne bajty (`byte_enable`).
2. Zaprojektuj moduł oraz opracuj symulację układu implementującego pamięć ROM $1024 \times 32b$ z asynchronicznym odczytem.